

Q1:

Raw audio only shows amplitude over time. Spectrograms and MFCCs make the frequency patterns of speech easier for the model to recognize. MFCCs also compress the audio into a smaller input, which is better for a TinyML device.

Q2:

The representative dataset gives the converter sample inputs so it can estimate the normal value ranges in the model. It uses those ranges to convert floating-point values into INT8 values while trying to keep the model accurate.

Q3:

`audio_processor.get_data()` loads a batch of audio samples, processes them into the features expected by the model, and returns the feature data with the correct labels. It can also apply things like background noise and time shifting.

Q4:

`g_model` contains the trained and quantized model as a C/C++ byte array, while `g_model_len` stores its size. They must be copied correctly so the Arduino uses the exact model we trained in the notebook.

```
Heard unknown (246) @279120ms
Heard unknown (204) @282064ms
Heard unknown (201) @284560ms
Heard unknown (209) @287616ms
Heard unknown (248) @289184ms
Heard yes (201) @290096ms
Heard unknown (209) @291856ms
Heard unknown (243) @293424ms
```

Model hearing “yes” keyword